TAXI MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

P Sharweshvar

2116240701491

in partial fulfilment of the End Semester Examination for the course

CB23F35 –

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF ENGINEERING
COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

NOVEMBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled "**TAXI MANAGEMENT SYSTEM**" is the bonafide work of **P Sharweshvar [2116240701491]**, who carried out the individual project work on the Zoho Creator Low-Code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator during the academic year 2026–2027, Semester 5, under the supervision of Dr. Jinu Sophia J, Associate Professor, Department of Computer Science and Engineering.

SIGNATURE

Dr. Manoranjini J
HEAD OF THE DEPARTMENT
Department of Computer Science
and Engineering
Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

Dr. Jinu Sophia J
COURSE FACULTY
Associate Professor
Department of Computer Science
and Engineering
Rajalakshmi Engineering College

Submitted for the End Semester Examination held on: _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I, P Sharweshvar [2116240701491], hereby declare that the project report titled “Taxi Management System” submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report — including its data model, forms, reports, pages, Deluge scripts, workflows and schedules — was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zia or the AI application builder), code snippet or external resource used in this work has been duly acknowledged in the report.

Place: Chennai

Date: _____

SIGNATURE OF THE STUDENT
P Sharweshvar
[2116240701491]

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman Thiru. S. Meganathan, Chairperson Dr. (Mrs.) Thangam Meganathan and Vice-Chairman Mr. Abhay Meganathan for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal Dr. S.N. Murugesan, Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank Dr. Manoranjini J, Head of the Department, Department of Computer Science and Engineering for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty, Dr. Jinu Sophia J, Associate Professor, for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Creator Education Program for providing access to the Zoho Creator platform and learning resources, and I thank my family and friends for their unwavering support.

P Sharweshvar

ABSTRACT

Taxi operators handle a continuous flow of ride requests, driver and vehicle allocations, trips, fare collections and vehicle servicing. When these are managed through phone calls, paper registers and separate spreadsheets, bookings are recorded incompletely, the availability of drivers and vehicles is unclear, payments remain pending without follow-up and scheduled maintenance is missed. There is no single place where an operator can see the state of the fleet.

The Taxi Management System is a centralised low-code application developed using Zoho Creator to manage taxi operations. It includes seven linked forms — Customers, Drivers, Vehicles, Bookings, Trip Details, Payments and Driver Maintenances — and seven list reports that present the records of each form. The supplied application data contains five customers, four drivers, three vehicles, five bookings, three trips, four payments and three maintenance records, and these records are used in this report to illustrate the data model and to analyse data quality. A set of Deluge workflows for status control, driver and vehicle allocation, payment reconciliation and maintenance reminders is specified as a design; it is not part of the supplied application data.

The report documents the requirements, design, implementation, test plan and governance of the application, and it records the data-quality observations found in the supplied records, such as missing fare amounts, booking and payment statuses that disagree, and a maintenance task that has passed its scheduled date. Automation, notification channels, role-based portal profiles, dashboards and production deployment are identified as areas for further implementation and verification.

Keywords: Low-Code Development; Zoho Creator; Deluge Scripting; Workflow Automation; Fleet Management; Taxi Booking; Role-Based Access Control.

PROJECT AT A GLANCE

Attribute	Details
Project Title	Taxi Management System
Application Domain	Urban and regional passenger transport; taxi booking, trip, payment and fleet maintenance management (Chennai, Madurai, Srirangam and Cuddalore addresses in the sample data)
Student Name & Register No.	P Sharweshvar 2116240701491
Development Platform	Zoho Creator low-code platform
Application Scope	Customer, driver and vehicle administration; booking entry and status tracking; trip start and end times; payments by cash, UPI and card; vehicle maintenance scheduling and cost; reports
Forms	7 forms: Customers, Drivers, Vehicles, Bookings, Trip Details, Payments and Driver Maintenances
Reports	7 list reports: All Customers, All Drivers, All Vehicles, All Bookings, All Trip Details, All Payments and All Driver Maintenances
Sample Data Supplied	5 customers, 4 drivers, 3 vehicles, 5 bookings, 3 trips, 4 payments and 3 maintenance records
Automation	Workflows, schedules and Deluge functions are specified in this report as a design (status synchronisation, allocation checks, payment reconciliation, maintenance reminders). They are not included in the supplied data and must be verified in the live application
Process Control	Observed status values: Booking (Pending, Confirmed, Completed); Trip (Scheduled, In Progress, Completed); Payment (Pending, Paid); Maintenance (Scheduled, Completed); Driver (Available, On Trip, Inactive); Vehicle (Available)
Access Control	Designed profiles for Admin, Dispatcher, Driver and Customer; not evidenced in the supplied data
External Integration Points	None evidenced. Email, SMS, WhatsApp and payment-gateway integrations are proposed enhancements
Deployment Status	Production URL and deployment evidence not supplied

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	5
	PROJECT AT A GLANCE	6
	LIST OF TABLES	9
	LIST OF FIGURES	10
	LIST OF ABBREVIATIONS	11
1	**INTRODUCTION** 1.1 Background 1.2 Problem Statement 1.3 Objective 1.4 Scope of the Project 1.5 Justification for a Low-Code Approach 1.6 Organisation of the Report	12
2	**LITERATURE SURVEY AND SYSTEM STUDY** 2.1 Study of the Existing System 2.2 Limitations of the Existing System 2.3 Low-Code versus Traditional Development 2.4 Literature Survey 2.5 Emerging Trends in Low-Code Development 2.6 Proposed System	15
3	**REQUIREMENT ANALYSIS** 3.1 Stakeholders and User Roles 3.2 Functional Requirements 3.3 Non-Functional Requirements 3.4 Hardware and Software Requirements 3.5 Use Case Model	18
4	**SYSTEM DESIGN** 4.1 System Architecture 4.2 Data Model 4.3 Form Design 4.4 Process and Workflow Design 4.5 User Interface and Navigation Design 4.6 Access Control Design	22

CHAPTER NO.	TITLE	PAGE NO.
5	**IMPLEMENTATION** 5.1 Application Setup 5.2 Forms and Data Capture 5.3 Reports 5.4 Pages and Dashboards 5.5 Deluge Scripting 5.6 Workflows and Automation	27
6	**TESTING AND DEBUGGING** 6.1 Testing Strategy 6.2 Test Cases 6.3 Debugging Deluge Scripts 6.4 Defect Log 6.5 User Acceptance Testing	32
7	**DEPLOYMENT, SECURITY AND GOVERNANCE** 7.1 Environments and Deployment Strategy 7.2 Performance Optimisation 7.3 Security Features 7.4 Governance and Administration	36
8	**RESULTS AND DISCUSSION** 8.1 Application Output 8.2 Achievement of Objectives 8.3 Comparison with the Existing System 8.4 Limitations of the Low-Code Approach 8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum	38
9	**CONCLUSION AND FUTURE ENHANCEMENTS** 9.1 Conclusion 9.2 Future Enhancements 9.3 Learning Outcomes and Career Relevance	41
10	REFERENCES	42
11	APPENDIX A – FEATURE COVERAGE CHECKLIST	43
12	APPENDIX B – REPORTS AND PAGES CATALOGUE	44
13	APPENDIX C – DELUGE LOGIC SUMMARY	45
14	APPENDIX D – USER MANUAL	46
15	APPENDIX E – EVALUATION RUBRIC	48
16	APPENDIX F – VIVA-VOCE RECORD SHEET	49

LIST OF TABLES

Table 1.1: Project Objectives and Measures 13
Table 2.1: Low-Code and Traditional Development Comparison 16
Table 2.2: Literature Survey Sources 17
Table 3.1: Stakeholders and User Roles 18
Table 3.2: Functional Requirements 19
Table 3.3: Non-Functional Requirements 20
Table 3.4: Hardware and Software Requirements 20
Table 3.5: Use Case Descriptions 21
Table 4.1: Entities and Relationships 23
Table 4.2: Form Design Groups 24
Table 4.3: Field Specification – Bookings 24
Table 4.4: Field Specification – Trip Details and Payments 25
Table 4.5: Access Control Design 26
Table 5.1: Forms Implemented 27
Table 5.2: Reports Implemented 28
Table 5.3: Pages Implemented (Proposed Design) 30
Table 5.4: Deluge Logic Summary (Proposed Design) 31
Table 5.5: Workflow Summary (Proposed Design) 31
Table 6.1: Test Cases 33
Table 6.2: Defect and Validation Log 34
Table 7.1: Deployment Checklist 36
Table 7.2: Security Controls 37
Table 8.1: Operational Snapshot from the Supplied Data 38
Table 8.2: Objective Achievement Mapping 39
Table 8.3: Comparison with the Existing System 40
Table A.1: Feature Coverage Checklist 43
Table B.1: Scheduled Monitoring 44
Table D.1: Troubleshooting 47
Table E.1: Evaluation Rubric for the Individual Project 48
Table E.2: Performance Level Descriptors 48
Table F.1: Viva-Voce Questions and Responses 49
Table F.2: Marks Summary 49

LIST OF FIGURES

Figure 2.1: Taxi Management System process represented as the proposed controlled workflow 15

Figure 3.1: Primary actors and the Taxi Management System 21

Figure 4.1: Logical architecture of the Taxi Management System 22

Figure 4.2: Conceptual relationships between core Taxi Management System entities 23

Figure 4.3: Status lifecycles of bookings, trips, payments and maintenance 25

Figure 5.1: All Customers report (sample data as supplied) 28

Figure 5.2: All Drivers report (sample data as supplied) 28

Figure 5.3: All Vehicles report (sample data as supplied) 28

Figure 5.4: All Bookings report (sample data as supplied; empty Fare_Amount means no value stored) 29

Figure 5.5: All Trip Details report (sample data as supplied) 29

Figure 5.6: All Payments report (sample data as supplied) 29

Figure 5.7: All Driver Maintenances report (sample data as supplied) 29

Figure 7.1: Environments 36

Figure 8.1: Payment amount by method and status 38

Figure 8.2: Maintenance cost by record and status 39

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
CRUD	Create, Read, Update and Delete
ER	Entity–Relationship
GPS	Global Positioning System
INR	Indian Rupee
KPI	Key Performance Indicator
LCAP	Low-Code Application Platform
RBAC	Role-Based Access Control
TMS	Taxi Management System
UAT	User Acceptance Testing
UI	User Interface
UPI	Unified Payments Interface
URL	Uniform Resource Locator

CHAPTER 1
INTRODUCTION

1.1 Background

A taxi service depends on a steady sequence of short, time-sensitive decisions. A customer asks for a pickup at a given place and time; the operator must decide which driver and vehicle can take the trip, note the fare, follow the trip to completion, collect the payment and keep every vehicle fit for the road. When these decisions are recorded in paper registers, personal spreadsheets and call logs, the operator cannot easily tell which driver is free, which vehicle is due for service or which payments are still outstanding.

The Taxi Management System brings customer, driver and vehicle records, bookings, trips, payments and vehicle maintenance together in one centralised Zoho Creator application. The sample records supplied with the application contain customer addresses in Chennai and Cuddalore and booking routes that include Srirangam, Madurai, Koyambedu and Anna Nagar, which reflects the intended operating area of a regional taxi operator.

1.2 Problem Statement

Unstructured booking and fleet records make it difficult to know which rides are confirmed, who is driving which vehicle, what has been paid and which vehicles need service. The following problems were identified from the study of the existing process and from the supplied sample records.

- Bookings are noted by phone or on paper and are often incomplete; in the sample data, three of the five bookings carry no fare amount.
- The availability of drivers and vehicles is not visible, so a driver or vehicle may be promised to two trips at the same time.
- Trip start and end times are recorded loosely, so trip duration cannot be measured and trips cannot be audited.
- Payments stay pending without follow-up and are not reconciled with the fare of the booking.
- Routine services and repairs are not scheduled against a date; a service that passes its date goes unnoticed.
- Customers are not distinguished by type (regular, corporate or new), so service levels and billing cannot be differentiated.

1.3 Objectives

- Design linked data forms for customers, drivers, vehicles, bookings, trips, payments and maintenance.
- Capture bookings with pickup and drop locations, date, pickup time, fare and status.
- Allocate an available driver and vehicle to a confirmed booking and record the actual trip start and end times.
- Record payments by method and status and reconcile them with booking fares.
- Schedule vehicle maintenance, record its cost and track its completion.
- Provide reports, and a set of designed automation rules, that give the operator a live view of bookings, fleet availability, payments and maintenance.

Objective ID	Objective	Evidence / Output
OBJ-01	Design linked data forms for customers, drivers, vehicles, bookings, trips, payments and maintenance.	Seven forms evidenced by the supplied reports
OBJ-02	Capture bookings with locations, date, time, fare and status.	All Bookings report: 5 records
OBJ-03	Allocate driver and vehicle; record trip start and end times.	All Drivers, All Vehicles and All Trip Details reports
OBJ-04	Record payments by method and status; reconcile with fares.	All Payments report: 4 records
OBJ-05	Schedule maintenance and track cost and completion.	All Driver Maintenances report: 3 records
OBJ-06	Provide reports and designed automation for live visibility.	Seven list reports; workflow design in Chapter 5

Table 1.1: Project Objectives and Measures

1.4 Scope of the Project

The project scope covers the administration of customers, drivers and vehicles; booking entry with pickup and drop locations, booking date, pickup time and fare; booking status tracking; trip records with actual start and end times; payments recorded by date, amount, method and status; vehicle maintenance records with type, date, cost and status; and seven list reports, one for each form.

The following items are described in the design but are not evidenced in the supplied application data: workflows and Deluge scripts, scheduled workflows, dashboard and home pages, customer portal profiles, email or SMS notifications, automatic fare calculation, GPS tracking, online payment gateways and a driver mobile application. They are presented as proposed design in Chapters 4 and 5 or as future enhancements in Chapter 9, and they must be verified in the live application before they are claimed as delivered.

1.5 Justification for a Low-Code Approach

Zoho Creator provides visual form building, lookup relationships, reports, pages, customer portals, workflow configuration and Deluge scripting within one platform. The Taxi Management System is process-oriented: most requirements involve data capture, validation, status changes, reconciliation and reminders rather than a completely custom user interface. Low-code development shortens repetitive implementation work and makes routine changes to forms, reports and workflows easier to maintain. Platform capabilities and limitations are discussed with reference to the official Zoho Creator documentation [1].

1.6 Organisation of the Report

Chapter 2 studies the existing process and the proposed solution. Chapter 3 presents stakeholders and requirements. Chapter 4 describes architecture, data relationships and process design. Chapter 5 documents the application forms, reports and the specified automation. Chapter 6 presents the test approach, test cases and the data-quality log. Chapter 7 discusses deployment, security and governance. Chapter 8 reviews the outcomes and limitations. Chapter 9 concludes the report and identifies future enhancements.

CHAPTER 2
LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

In the existing scenario, a customer phones the operator or a driver, or sends a message, to ask for a ride. The booking is written in a register or a spreadsheet, usually without a fare, and the dispatcher assigns a driver by phone from memory. Trip times are noted by the driver, if at all. Payment is collected in cash, by UPI or by card and is entered later, if the trip is remembered. Service dates for vehicles are tracked informally, and the operator learns of a missed service only when a vehicle breaks down.

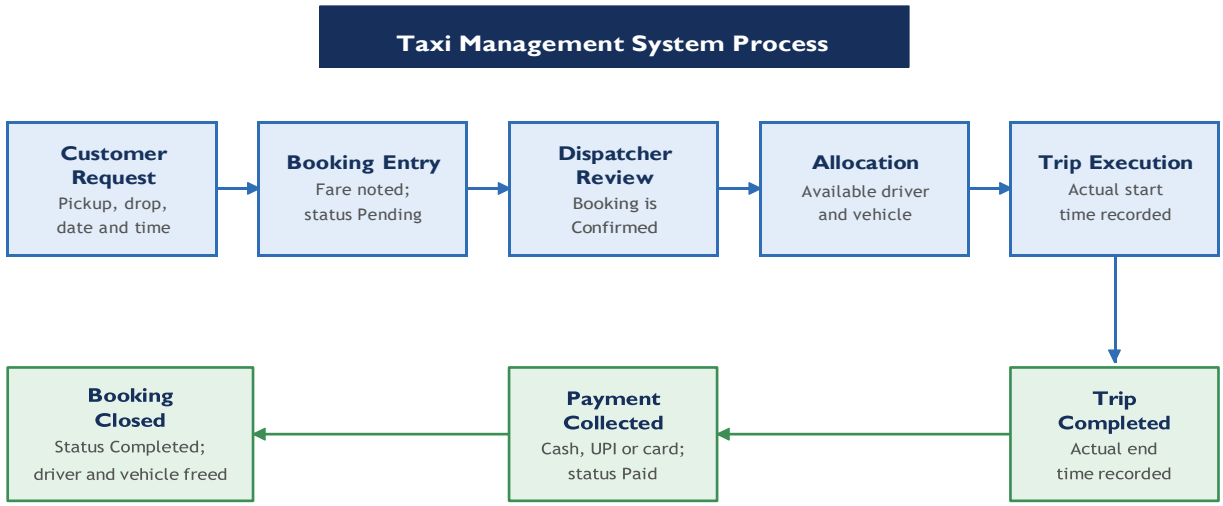


Figure 2.1: *Taxi Management System process represented as the proposed controlled workflow*

2.2 Limitations of the Existing System

- Booking details are incomplete or inconsistent, so fares, locations and times cannot be relied on.
- Driver and vehicle availability is not visible, which causes double allocation or idle vehicles.
- Trip start and end times are not captured, so duration and punctuality cannot be measured.
- Payments are not linked to bookings, so outstanding amounts are discovered late.
- Maintenance is not scheduled or tracked against dates and costs.
- Management reports require manual consolidation and may be out of date.

2.3 Low-Code versus Traditional Development

Criterion	Traditional Pro-Code Development	Low-Code on Zoho Creator
Development effort	Requires a custom UI, database, APIs and deployment setup.	Uses visual form, report and page builders with configured workflows.
Data capture	Forms and validation are coded individually.	Forms, fields, lookups and subforms are configured visually.
Automation	Requires application logic and scheduled jobs to be implemented.	Workflow rules, schedules and Deluge scripts support event-driven automation.
Access for users	Requires custom authentication and role management.	Customer portal and user profiles with configurable permissions are available.
Reports	Reporting screens and exports are built separately.	List reports with filters, search and export are generated from forms.
Hosting	Infrastructure and release process are managed separately.	Platform hosting and application publishing are provided by the service.
Customisation	Offers broad control over architecture and UI.	Customisation is bounded by platform capabilities and plan limits.
Maintenance	Requires code-level maintenance and deployment discipline.	Routine changes may be simpler, but scripts and permissions still need testing.

Table 2.1: Low-Code and Traditional Development Comparison

2.4 Literature Survey

The following sources provide the platform and process background for this project. The supplied project data does not include a formal research-paper survey; the table therefore focuses on authoritative platform documentation, established ride-booking platforms and a security reference rather than claiming results from experiments that were not carried out.

S.No.	Source	Focus	Relevance
1	Zoho Creator documentation [1]	Forms, reports, pages and application building	Primary reference for platform capabilities.
2	Zoho Deluge documentation [2]	Scripting language and task syntax	Supports validation, allocation and automation logic.
3	Zoho Creator workflow documentation [3]	Form-event workflow rules	Relevant to booking, trip, payment and maintenance triggers.
4	Zoho Creator schedules documentation [4]	Recurring and scheduled actions	Relevant to maintenance reminders and pending-payment follow-up.
5	Zoho Creator customer portal documentation [5]	Portal users and profiles	Relevant to customer, driver and dispatcher access.
6	Uber [6]	On-demand ride booking	Reference model for booking, trip status and fare.
7	Ola Cabs [7]	Taxi booking in Indian cities	Reference model for driver and vehicle allocation.
8	OWASP Application Security Verification Standard [8]	Application security controls	General checklist for access and data protection.

Table 2.2: Literature Survey Sources

2.5 Emerging Trends in Low-Code Development

Low-code platforms increasingly combine visual modelling, reusable components, event-driven automation, integrations and analytics. For the Taxi Management System, these patterns support a single operational data model, repeatable status changes and fleet-wide reporting. Automated checks should still be designed carefully: workflows need duplicate prevention, clear failure handling, role-appropriate permissions and testing with both valid and invalid records. Ride-booking platforms also show the value of live trip status, digital payment records and proactive vehicle maintenance, which this project addresses through linked trip, payment and maintenance forms.

2.6 Proposed System

The proposed system centralises taxi operations in linked Zoho Creator forms. A booking is entered with its locations, time and fare; once confirmed, an available driver and vehicle are allocated and the driver status changes to On Trip. The trip record captures actual start and end times; completing the trip closes the booking and releases the driver. Each booking has a payment record whose method and status are tracked until the fare is paid. Maintenance records capture the type, scheduled date, cost and status of each service, and a daily schedule flags services that are due or overdue. Seven reports provide visibility across the fleet.

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Role	Responsibilities
Customer	Request rides, give pickup and drop details, track booking status, pay the fare and review trip history. Customers are classified as Regular, Corporate or New.
Dispatcher	Create and confirm bookings, allocate an available driver and vehicle, monitor trips and follow up pending payments.
Driver	View assigned trips, start and end trips, keep duty status current (Available, On Trip or Inactive) and report vehicle issues.
Admin / Fleet Manager	Maintain customers, drivers and vehicles; schedule and close maintenance; review reports; manage users and access.

Table 3.1: Stakeholders and User Roles

3.2 Functional Requirements

ID	Requirement	Role(s)	Priority
FR-01	Register customers with name, phone, email, address and customer type (Regular, Corporate, New).	Admin / Dispatcher	High
FR-02	Register drivers with phone number, driving licence number, joining date and status (Available, On Trip, Inactive).	Admin	High
FR-03	Maintain vehicles with name, registration number, type (Sedan, SUV) and status.	Admin	High
FR-04	Create a booking with pickup location, drop location, booking date, pickup time and fare amount.	Dispatcher / Customer	High
FR-05	Track booking status through Pending, Confirmed and Completed.	Dispatcher	High
FR-06	Allocate only an Available driver and an Available vehicle to a confirmed booking and set the driver to On Trip.	System	High
FR-07	Record a trip against a booking with actual start time, actual end time and status (Scheduled, In Progress, Completed); reject an end time earlier than the start time.	Driver / System	High
FR-08	Record a payment against a booking with date, amount, method (Cash, UPI, Card) and status (Pending, Paid).	Dispatcher	High
FR-09	Compare the payment amount with the booking fare and flag a mismatch; keep the booking and payment statuses consistent.	System	Medium
FR-10	Record maintenance with type, service date, cost and status (Scheduled, Completed).	Admin	High
FR-11	Flag maintenance whose scheduled date has passed and keep a vehicle out of allocation while it is in service.	System	Medium
FR-12	Prevent allocation of Inactive or On Trip drivers and of vehicles that are not Available.	System	High
FR-13	Notify the customer by email when a booking is confirmed and when the trip is completed.	System	Medium
FR-14	Provide a list report for each form with search, filter and export for authorised roles.	All roles	Medium
FR-15	Provide dashboard counters for pending bookings, available drivers and vehicles, pending payment amount and maintenance due.	Dispatcher / Admin	Medium
FR-16	Restrict forms, reports and actions by role using least-privilege permissions.	Admin	High

Table 3.2: Functional Requirements

3.3 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Data integrity	Lookup relationships, unique fields (phone, email, licence number, vehicle number) and validation should keep linked records consistent.
NFR-02	Security and privacy	Customer phone numbers, emails and driver licence numbers should be visible only to roles that need them.
NFR-03	Usability	Booking entry should take only a few fields and work on a mobile browser; locations should be entered consistently.
NFR-04	Maintainability	Status values and thresholds (maintenance reminder days, pending-payment ageing) should be configured, not hard-coded.
NFR-05	Auditability	Trip times, payments and maintenance should retain traceable records with date and user.
NFR-06	Availability	Application access depends on platform availability and network connectivity.
NFR-07	Performance	Reports and counters should filter data appropriately and avoid unnecessary repeated queries.

Table 3.3: Non-Functional Requirements

3.4 Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator low-code application platform
Scripting	Deluge for form validation, workflow automation, scheduled logic and notifications
Client	Desktop, laptop or smartphone with a modern browser and a stable internet connection
Browser	Current version of Google Chrome, Microsoft Edge or Mozilla Firefox
Development	Zoho Creator form, report, page, workflow and schedule builders
Optional integration tools	Email, SMS or WhatsApp providers and a payment gateway; credentials and endpoint tests are required before live use

Table 3.4: Hardware and Software Requirements

3.5 Use Case Model

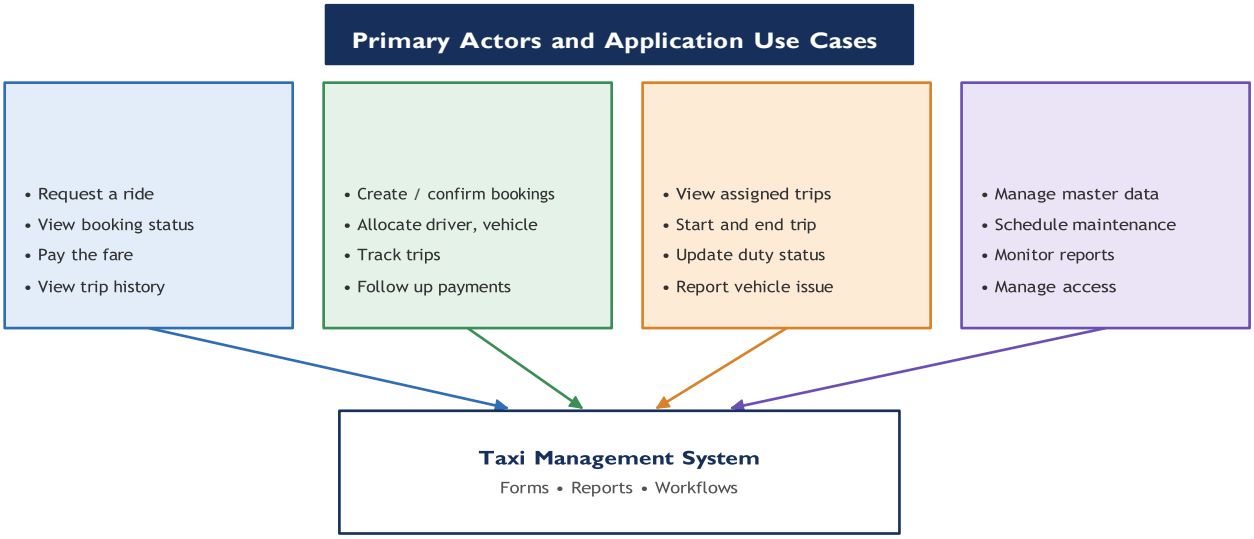


Figure 3.1: *Primary actors and the Taxi Management System*

Use Case	Precondition	Main flow	Result
Create booking	Customer record exists.	Enter pickup, drop, date, time and fare and save.	Booking is stored with status Pending.
Confirm and allocate	Booking is Pending; a driver and a vehicle are Available.	Confirm the booking and select driver and vehicle.	Booking is Confirmed; driver becomes On Trip.
Run trip	Booking is Confirmed and allocated.	Driver starts the trip and later ends it.	Actual start and end times are stored; trip is Completed.
Collect payment	Booking has a fare.	Record date, amount, method and status.	Payment is Paid or Pending; booking status is kept consistent.
Maintain vehicle	Vehicle exists.	Schedule a service with date and cost; mark Completed after service.	Maintenance history and cost are available; overdue services are flagged.

Table 3.5: Use Case Descriptions

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

The architecture separates the user-facing layer, the automation layer and the stored application records. Dispatchers, drivers, customers and administrators interact with Zoho Creator forms, reports and pages. Deluge workflows, form-event validations and daily schedules are designed to enforce the process rules. Linked records provide the operational data used by the reports and dashboard counters.

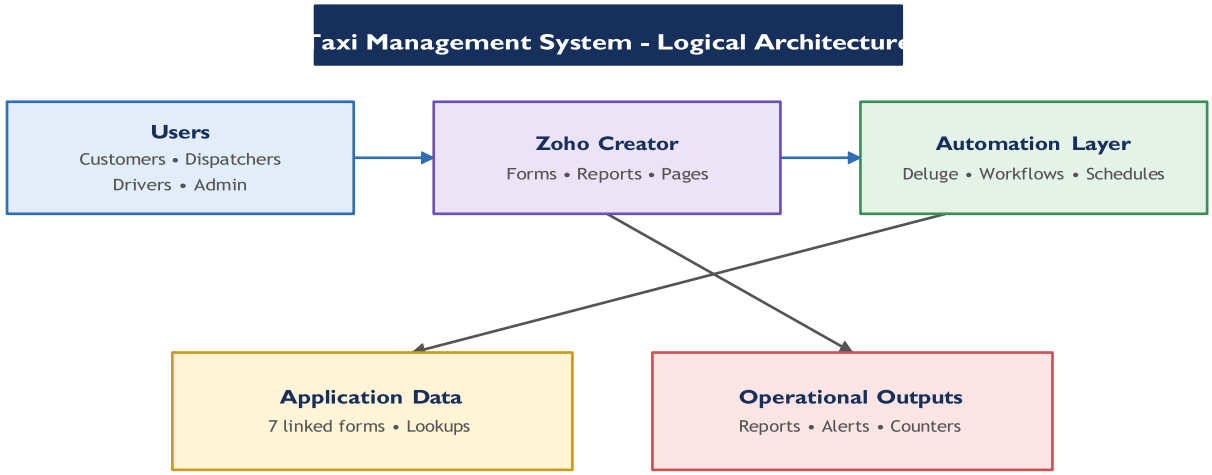


Figure 4.1: *Logical architecture of the Taxi Management System*

4.2 Data Model

The data model uses master records for relatively stable information (Customers, Drivers and Vehicles) and transaction records for operations (Bookings, Trip Details, Payments and Driver Maintenances). Lookup fields link transactions to master records. Booking is the central transaction: each booking belongs to a customer and is served by a driver and a vehicle, and each trip and each payment points to a booking. A maintenance record is linked to the vehicle that is serviced. The exported reports show only the stored data columns; the lookup columns that implement these links are part of the design and should be confirmed in the form builder of the live application.

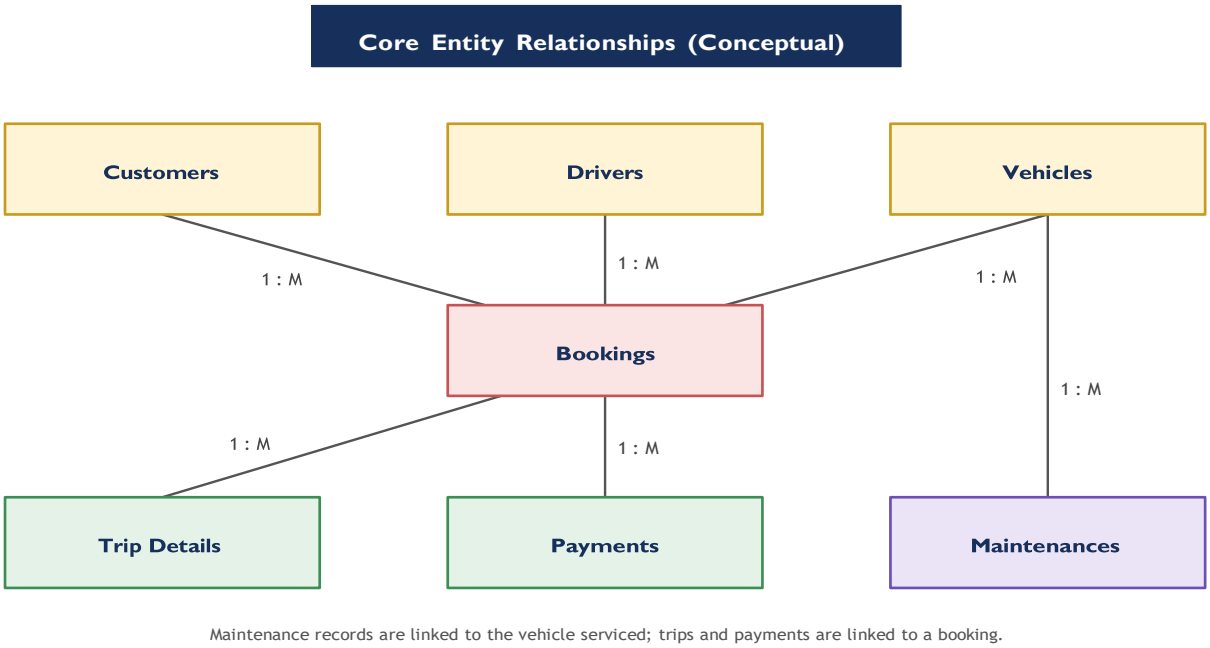


Figure 4.2: *Conceptual relationships between core Taxi Management System entities*

Entity	Relationship / Purpose
Customers	Passengers with phone, email, address and a type (Regular, Corporate or New). One customer can make many bookings.
Drivers	Drivers with phone number, driving licence number, joining date and status. One driver can serve many bookings.
Vehicles	Fleet vehicles with name, registration number, type and status. One vehicle can serve many bookings and have many maintenance records.
Bookings	Central transaction: pickup and drop locations, booking date, pickup time, fare and status.
Trip Details	Execution of a booking: actual start time, actual end time and trip status.
Payments	Money collected for a booking: date, amount, method and status.
Driver Maintenances	Service history and schedule: maintenance type, service date, cost and status.

Table 4.1: Entities and Relationships

4.3 Form Design

Form group	Design considerations
Master forms	Customers, Drivers and Vehicles store reusable reference data; unique constraints on phone, email, licence number and registration number prevent duplicates.
Booking forms	Bookings and Trip Details are kept short; customer, driver and vehicle are selected from lookups rather than typed.
Finance forms	Payments hold amount, method and status; the amount is compared with the booking fare.
Fleet care forms	Driver Maintenances records the type, date, cost and status of each service and drives reminders.

Table 4.2: Form Design Groups

The principal fields of the transaction forms are specified below. Field names follow the column headings of the supplied reports; fields marked (design) are lookups or rules that are part of the proposed design.

Field	Type	Rule / Notes
Booking_ID	Auto number	Unique booking identifier.
Customer (design)	Lookup (Customers)	Mandatory; selects the customer who made the booking.
Pickup_Location	Text / Address	Mandatory; use consistent spelling and capitalisation.
Drop_Location	Text / Address	Mandatory.
Booking_Date	Date	Format dd-MMM-yyyy.
Pickup_Time	Date-time	24-hour format.
Booking_Status	Drop-down	Pending, Confirmed, Completed (Cancelled proposed).
Fare_Amount	Decimal (₹)	Should be mandatory before the status becomes Confirmed; empty in three sample records.
Driver, Vehicle (design)	Lookup	Set at allocation; only Available records may be selected.

Table 4.3: Field Specification – Bookings

Form	Field	Type	Rule / Notes
Trip Details	Trip_ID	Auto number	Unique trip identifier; linked to one booking (design).
Trip Details	Actual_Start_Time	Date-time	Set when the trip starts.
Trip Details	Actual_End_Time	Date-time	Set when the trip ends; must not be earlier than the start time.
Trip Details	Trip_Status	Drop-down	Scheduled, In Progress, Completed.
Payments	Payment_ID	Auto number	Unique payment identifier; linked to one booking (design).
Payments	Payment_Date	Date	Format dd-MMM-yyyy.
Payments	Amount	Decimal (₹)	Compared with the booking fare.
Payments	Payment_Method	Drop-down	Cash, UPI, Card.
Payments	Payment_Status	Drop-down	Pending, Paid.

Table 4.4: Field Specification – Trip Details and Payments

4.4 Process and Workflow Design

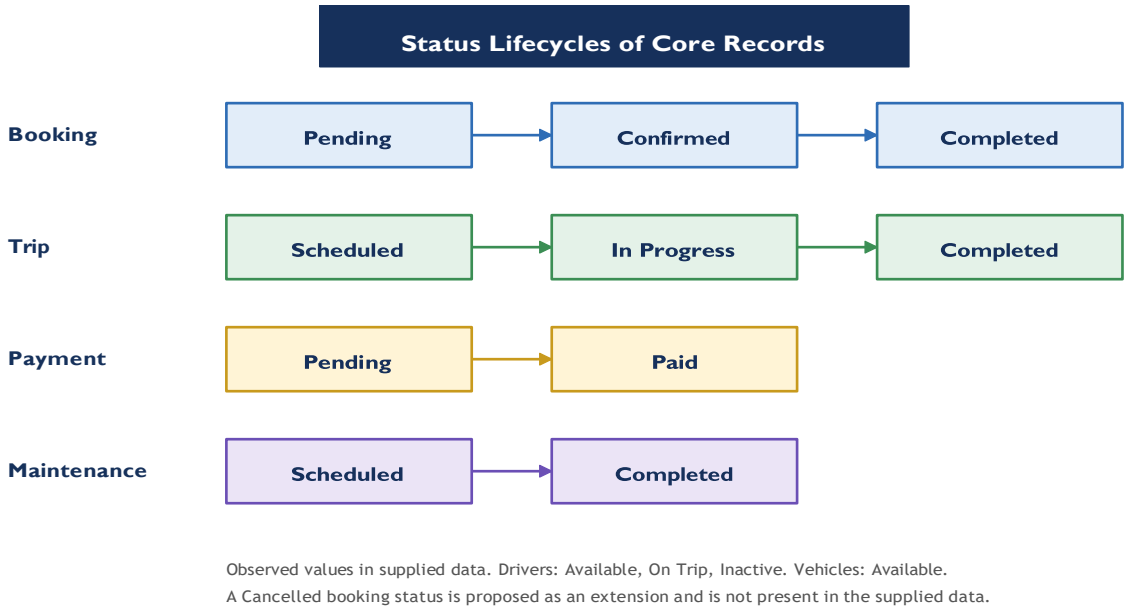


Figure 4.3: *Status lifecycles of bookings, trips, payments and maintenance*

The lifecycle of a ride is controlled through the status fields of four forms. A booking starts as Pending, becomes Confirmed when the dispatcher confirms it and allocates a driver and vehicle, and becomes Completed when the trip ends. A trip moves from Scheduled to In Progress when the driver starts it and to Completed when it ends, and the actual end time must not precede the actual start time. A payment moves from Pending to Paid when the fare is collected. A maintenance record moves from Scheduled to Completed when the service is done. Driver status follows the trip: Available before allocation, On Trip while a trip is In Progress, and Available again after completion; Inactive drivers are excluded from allocation.

The supplied data does not show whether Zoho Creator Blueprint or workflows are used. This report therefore treats the enforcement of the lifecycles as a design that is implemented through form validation and workflows and is to be verified in the live application.

4.5 User Interface and Navigation Design

The application is organised around role-specific tasks. Dispatchers need a quick route to create bookings, see which drivers and vehicles are available and follow trips and payments. Drivers need to see only their own assigned trips. Administrators need the master-data, maintenance and report views. A dashboard page is proposed with counters for pending bookings, available drivers, available vehicles, pending payment amount and maintenance due, followed by the Bookings report. The supplied application data lists seven reports; pages are described in Chapter 5 as a proposed design. Status values and colours should be kept consistent across reports and pages.

4.6 Access Control Design

Role	Typical permitted actions	Restricted / controlled actions
Customer	Create bookings (portal); view own bookings, trips and payments.	Cannot view other customers, driver details, other bookings or maintenance.
Dispatcher	Create and edit bookings; allocate driver and vehicle; view trips; record payments; view customers, drivers and vehicles.	Cannot delete records, edit maintenance cost or change user access.
Driver	View assigned trips; start and end trips; update own duty status.	Cannot edit payments, fares or other drivers' trips.
Admin	Manage customers, drivers, vehicles and maintenance; view all reports; manage users and access.	Privileged activity should be limited and reviewed.

Table 4.5: Access Control Design

CHAPTER 5
IMPLEMENTATION

5.1 Application Setup

The application is built on Zoho Creator using visual builders for forms and reports. The supplied reports show the date format dd-MMM-yyyy, a 24-hour time format and amounts in Indian rupees (₹). A consistent naming scheme is used for forms and fields (for example Booking_ID, Pickup_Location and Fare_Amount) because Deluge scripts refer to field link names. Before publishing changes, the developer should verify field link names, lookup targets, mandatory fields and workflow execution order.

5.2 Forms and Data Capture

No.	Form	Purpose and principal fields
1	Customers	Customer name, phone number, email, address and customer type (Regular, Corporate, New).
2	Drivers	Driver name, phone number, driving licence number, driver status (Available, On Trip, Inactive) and joining date.
3	Vehicles	Vehicle name, vehicle number, vehicle type (Sedan, SUV) and vehicle status.
4	Bookings	Booking ID, pickup location, drop location, booking date, pickup time, booking status and fare amount.
5	Trip Details	Trip ID, actual start time, actual end time and trip status (Scheduled, In Progress, Completed).
6	Payments	Payment ID, payment date, amount, payment method (Cash, UPI, Card) and payment status (Pending, Paid).
7	Driver Maintenances	Maintenance ID, maintenance type, service date, service cost and maintenance status (Scheduled, Completed).

Table 5.1: Forms Implemented

5.3 Reports

Seven list reports provide a tabular view of each form. The data shown in the figures that follow is taken from the report exports supplied with the application and is sorted by identifier for readability. Together the figures also act as the sample data set used for the analysis in Chapters 6 and 8.

No.	Report	Use
1	All Customers	Review customers, contact details, addresses and customer types.
2	All Drivers	Review drivers, licences, status and joining dates.
3	All Vehicles	Review the fleet, vehicle numbers, types and status.
4	All Bookings	Review bookings with locations, time, status and fare.
5	All Trip Details	Review actual trip start and end times and trip status.
6	All Payments	Review payments by date, amount, method and status.
7	All Driver Maintenances	Review service type, date, cost and status.

Table 5.2: Reports Implemented

All Customers

Customers_Name	Phone_Number	Email	Address	Customer_Type
Suriya Sivan	+916485295326	ss234@gmail.com	60, Thiru VK Street, Pandi Area, Cuddalore	Corporate
Stephen Raj	+919658659598	stejar@gmail.com	Lion Estates, KK Colony, Chennai	New
Abdul Ravoof	+918165498659	abdrav@gmail.com	21C Crown Aparts, No 1 Tollgate, Chennai	Regular
Priya Sharma	+918642359578	priya321@gmail.com	F12 Green Meadows, Anna Nagar, Chennai	Corporate
Arun Kumar	+919265743158	arunk@gmail.com	90, NK Nagar, Velachery, Chennai	Regular

Figure 5.1: *All Customers report (sample data as supplied)*

All Drivers

Driver_Name	Driver_Phone_Number	Driving_License_No	Driver_Status	Driver_Joining_Date
Jesu Raj	+919858596363	TN 25147	On Trip	03-Jul-2024
Mari Selvan	+918975623798	TN 93626	Inactive	16-Sep-2022
Suresh Babu	+918649535674	TN 86759	Available	28-Aug-2023
Ravi Kumar	+919678541328	TN 25769	Available	08-Feb-2021

Figure 5.2: *All Drivers report (sample data as supplied)*

All Vehicles

Vehicle_Name	Vehicle_Number	Vehicle_Type	Vehicle_Status
Ciaz	TN 81 BN 3625	Sedan	Available
Toyota Innova	TN 48 AB 5678	SUV	Available
Swift Dzire	TN 45 CD 1234	Sedan	Available

Figure 5.3: *All Vehicles report (sample data as supplied)*

All Bookings

Booking_ID	Pickup_Location	Drop_Location	Booking_Date	Pickup_Time	Booking_Status	Fare_Amount
1	Srirangam Railway Station	Courtyard	18-Oct-2026	11:00:00	Confirmed	
2	Maattuthavani Bus Stop	Madurai Airport	19-Oct-2026	15:30:00	Completed	
3	tower park	koyambedu	15-Oct-2026	16:00:32	Completed	
4	PN hotel	Junction	07-Oct-2026	16:40:10	Pending	₹ 200.00
5	Anna Nagar	Banglore	21-Oct-2026	17:45:01	Pending	₹ 4,400.00

Figure 5.4: *All Bookings report (sample data as supplied; empty Fare_Amount means no value stored)*

All Trip Details

Trip_ID	Actual_Start_Time	Actual_End_Time	Trip_Status
1	18-Oct-2026 11:00:15	18-Oct-2026 11:10:02	In Progress
2	19-Oct-2026 15:30:52	19-Oct-2026 15:50:08	Completed
3	21-Oct-2026 17:00:36	23-Oct-2026 17:00:50	Scheduled

Figure 5.5: *All Trip Details report (sample data as supplied)*

All Payments

Payment_ID	Payment_Date	Amount	Payment_Method	Payment_Status
1	18-Oct-2026	₹ 600.00	UPI	Pending
2	19-Oct-2026	₹ 1,200.00	Cash	Paid
3	07-Oct-2026	₹ 200.00	Cash	Paid
4	21-Oct-2026	₹ 4,400.00	Card	Pending

Figure 5.6: *All Payments report (sample data as supplied)*

All Driver Maintenances

Maintenance_ID	Maintenance_Type	Service_Date	Service_Cost	Maintenance_Status
1	Regular Service	08-Oct-2026	₹ 950.00	Completed
2	Oil Replacement	04-Nov-2026	₹ 5,800.00	Scheduled
3	Tyre Replacement	30-Oct-2026	₹ 2,000.00	Completed

Figure 5.7: *All Driver Maintenances report (sample data as supplied)*

5.4 Pages and Dashboards

The supplied application data lists reports only and does not include page definitions. The pages below are therefore a proposed design that gives each role a simple starting point. They should be built with the Zoho Creator page builder and verified before they are claimed as delivered.

No.	Page	Proposed content
1	Dashboard	Counters for pending bookings, confirmed bookings, available drivers, available vehicles, pending payment amount and maintenance due, followed by the Bookings report.
2	Book a Ride	Embedded Bookings form with customer, pickup, drop, date and time, and a short guide.
3	My Bookings	Report filtered to the logged-in customer; editing and export disabled.
4	Driver Duty	Report filtered to the logged-in driver showing assigned trips with start and end actions.

Table 5.3: Pages Implemented (Proposed Design)

5.5 Deluge Scripting

Deluge is proposed for field validation, allocation checks, status synchronisation, reconciliation and reminders. Scripts should be attached to the correct form event (on load, on validate or on success), use the actual field link names, and guard against empty lookups and repeated execution. The principal scripts are summarised in Table 5.4; the logic is described in Appendix C. These scripts are not part of the supplied application data and must be verified in the live application.

No.	Trigger / Area	Logic summary
1	Bookings – on add (defaults)	Set Booking_Status to Pending; require pickup, drop, date and time; show an alert when a mandatory value is missing.
2	Bookings – on validate (fare check)	Require a fare amount before the status can be changed to Confirmed.
3	Bookings – on success (allocation)	When the status becomes Confirmed, allow only an Available driver and Available vehicle; set the driver to On Trip.
4	Trip Details – on validate	Reject an end time earlier than the start time; reject a second In Progress trip for the same driver.
5	Trip Details – on success	When the trip is Completed, set the booking to Completed and the driver to Available.
6	Payments – on validate	Compare the amount with the booking fare and alert on a mismatch.
7	Payments – on success	When the payment is Paid, record the payment date and keep the booking and payment statuses consistent.
8	Driver Maintenances – on success	When a service is Scheduled, keep the vehicle out of allocation on the service date; when Completed, release it.
9	Schedules (Maintenance Reminder, Payment Follow-up)	Flag maintenance whose date has passed; list payments pending for longer than the configured days.
10	Function tms.notifyCustomer	Send an email to the customer for booking confirmation and trip completion when an email address exists.

Table 5.4: Deluge Logic Summary (Proposed Design)

5.6 Workflows and Automation

The workflows below translate the Deluge logic into form and schedule workflows. They are proposed for configuration in the application. The supplied application data does not show which of them exist or whether they are active, so the status column records that every item is to be configured and verified.

No.	Workflow	Form / Type	Event	Status
1	Set Booking Defaults	Bookings	On add – on load	To be configured
2	Check Fare Before Confirm	Bookings	On edit – on validate	To be configured
3	Allocate Driver and Vehicle	Bookings	On edit – success	To be configured
4	Validate Trip Times	Trip Details	On add / edit – on validate	To be configured
5	Complete Trip	Trip Details	On edit – success	To be configured
6	Check Payment Amount	Payments	On add – on validate	To be configured
7	Sync Payment Status	Payments	On add / edit – success	To be configured
8	Update Vehicle Status	Driver Maintenances	On add / edit – success	To be configured
9	Maintenance Reminder	Schedule	Daily at 06:00 (Asia/Kolkata)	To be configured
10	Pending Payment Follow-up	Schedule	Daily at 06:00 (Asia/Kolkata)	To be configured

Table 5.5: Workflow Summary (Proposed Design)

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

Testing should cover individual field rules, workflow triggers, linked-record consistency and end-to-end booking scenarios. Each test should be performed with valid and invalid data and the result recorded. The cases below are a practical test plan based on the designed workflows; they should be marked Pass only after execution in the live application.

6.2 Test Cases

Test ID	Area	Test input / action	Expected result	Status
TC-01	Mandatory fields	Save a booking without pickup or drop location.	Save is blocked with an alert.	To be executed
TC-02	Unique values	Add a customer with an existing phone number or email.	Duplicate is rejected.	To be executed
TC-03	Booking defaults	Create a new booking.	Status is Pending.	To be executed
TC-04	Allocation	Confirm a booking and select an Available driver and vehicle.	Booking is Confirmed; driver becomes On Trip.	To be executed
TC-05	Unavailable driver	Select an Inactive driver for a booking.	Selection is rejected.	To be executed
TC-06	Vehicle conflict	Allocate a vehicle that is already on a trip.	Allocation is rejected.	To be executed
TC-07	Trip start	Start a trip.	Status becomes In Progress; start time is stored.	To be executed
TC-08	Trip completion	End the trip.	Trip and booking become Completed; driver becomes Available.	To be executed
TC-09	Invalid trip times	Enter an end time earlier than the start time.	Submission is rejected.	To be executed
TC-10	Payment entry	Add a payment for a booking.	Payment is stored as Pending or Paid.	To be executed
TC-11	Status sync	Mark a payment Paid.	Booking and payment statuses are consistent.	To be executed
TC-12	Fare mismatch	Enter a payment amount different from the fare.	An alert is shown.	To be executed
TC-13	Maintenance scheduling	Schedule a service for a vehicle.	Vehicle is kept out of allocation on the date.	To be executed
TC-14	Overdue maintenance	Run the reminder for a service dated in the past that is still Scheduled.	Record is flagged overdue.	To be executed
TC-15	Access control	Try to edit a payment as a driver.	Action is not available to the driver profile.	To be executed

Table 6.1: Test Cases

6.3 Debugging Deluge Scripts

When a workflow fails, first confirm the form event and execution timing. Verify field link names, lookup values and null handling. Use temporary diagnostic messages or logs during development, then remove sensitive debug output before deployment. Check whether another workflow updates the same field and whether the same event can execute more than once. Test status updates, notification creation and scheduled actions for repeated execution to prevent duplicate records or repeated emails. The execution history of each workflow shows the trigger, record, status and output of every run and is the first place to look when a script does not behave as expected.

6.4 Defect Log

The following data-quality observations were identified by reviewing the supplied report exports against each other. Records are related by date and amount because the key columns that link them are not shown in the list reports. Each observation is recorded for verification in the live application.

Defect ID	Observation in the supplied data	Resolution / control	Status
D-01	Booking 4 (fare ₹ 200.00, dated 07-Oct-2026) is Pending, while a ₹ 200.00 cash payment dated 07-Oct-2026 is Paid.	Synchronise booking status with payment and trip status through a workflow.	Verify in application
D-02	Fare_Amount is empty for bookings 1, 2 and 3, although bookings 2 and 3 are Completed.	Make the fare mandatory before the booking can be Confirmed.	Verify in application
D-03	Booking 3 is Completed but has no matching trip or payment; booking 4 has a payment but no trip.	Require a trip record before Completed and a payment record before closing.	Verify in application
D-04	Trip 1 is In Progress but already has an actual end time; trip 3 is Scheduled but has actual start and end times.	Fill actual times only through workflow on start and completion; add planned-time fields.	Verify in application
D-05	Locations are typed inconsistently (“Banglore”, “tower park”, “PN hotel”, “koyambedu”).	Use an address field or lookup list and consistent spelling and capitalisation.	Verify in application
D-06	Maintenance 2 (Oil Replacement, ₹ 5,800.00) was scheduled for 04-Nov-2026 and is still Scheduled after that date; its cost is high compared with the other records.	Add an overdue flag and reminder; confirm the cost entry.	Verify in application
D-07	Payments 1 (₹ 600.00, 18-Oct-2026) and 4 (₹ 4,400.00, 21-Oct-2026) remain Pending.	Add payment ageing and a follow-up schedule.	Verify in application
D-08	The All Payments report shows Save Changes and Remove Changes controls, which means inline editing is enabled.	Restrict inline editing to authorised roles.	Verify in application

Table 6.2: Defect and Validation Log

6.5 User Acceptance Testing

Acceptance testing should involve a customer, a dispatcher, a driver and an administrator following a complete scenario: add a customer, create a booking, confirm it and allocate a driver and vehicle, start and end the trip, record the payment, verify that statuses and dashboard counters are correct, and schedule and complete a vehicle service. Acceptance should be based on traceable records, correct statuses, appropriate access and clear status changes. No signed acceptance sheet was supplied with the application data.

CHAPTER 7
DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

The application should be developed and tested in a controlled development environment, promoted to a stage environment and then published to production. A release checklist should confirm that forms, lookup relationships, Deluge scripts, workflows, schedules, reports, pages and notification settings are configured in the target environment. The supplied data does not include a published URL or deployment record, so the final production deployment remains to be verified.

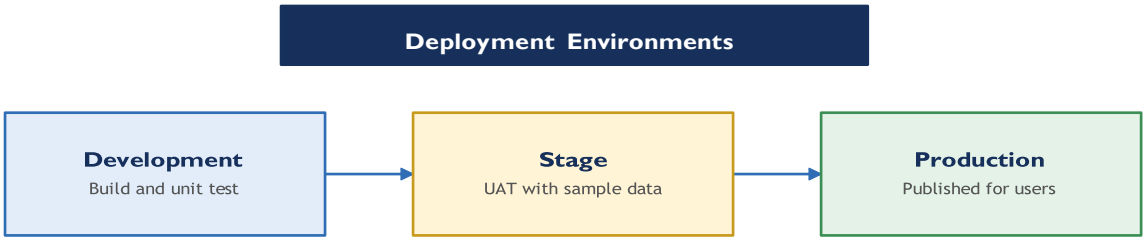


Figure 7.1: *Environments*

Deployment check	Verification
Data model	Confirm all forms, field link names and lookups.
Master data	Load customers, drivers (with licence numbers) and vehicles (with registration numbers).
Automation	Configure, activate and test workflows and schedules with sample records.
Notifications	Verify sender settings, recipient addresses and delivery outcomes.
Access	Test each role with least-privilege permissions.
Recovery	Define export, backup, recovery and change-control procedures.
Release	Record date, version, approver and known limitations.

Table 7.1: Deployment Checklist

7.2 Performance Optimisation

- Use report filters to limit data to relevant dates, statuses or drivers.
- Select drivers and vehicles from lookups filtered to Available records.
- Keep scheduled jobs daily and process only Scheduled maintenance and Pending payments.
- Avoid repeated updates or repeated notifications for the same record.
- Archive old Completed bookings, trips and payments as the volume grows.

7.3 Security Features

Control	Application of the control
Authentication	Recommended: users sign in through Zoho Creator, with the customer portal for customers.
Authorisation	Recommended: profiles for Admin, Dispatcher, Driver and Customer limit forms, reports and pages.
Privacy	Customer phone numbers and emails and driver licence numbers are personal data and should be visible only to roles that need them.
Validation	Unique constraints on phone, email, licence and vehicle numbers; mandatory fields; trip-time and fare checks.
Auditability	Trip times, payments and maintenance are stored as separate records with dates; add a log of status changes.
Edit control	Observed: inline editing controls appear in the All Payments report; restrict them to authorised roles.
Error handling	Workflows should use try and catch and email the administrator on failure; sensitive data should not be exposed in errors or logs.

Table 7.2: Security Controls

7.4 Governance and Administration

Governance covers who may change configuration, how changes are tested and how the application is maintained. Administrators should keep status values and thresholds in one place, review portal profile permissions periodically, record each release with its date and version, and keep a short log of changes to workflows and schedules. Weekly reviews of pending payments and overdue maintenance, and monthly reviews of trip volume and vehicle cost, help keep operational data reliable.

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Application Output

The application structure covers the taxi operation from customer and fleet records through booking, allocation, trip, payment and maintenance. The supplied records give the following operational snapshot as of the date of this report (10 November 2026).

Entity	Records	Observation from the supplied data
Customers	5	2 Corporate, 2 Regular and 1 New customer.
Drivers	4	2 Available, 1 On Trip and 1 Inactive; 50% of drivers can take a new trip.
Vehicles	3	2 Sedans and 1 SUV; all 3 are Available.
Bookings	5	2 Completed, 1 Confirmed and 2 Pending; fares are recorded for 2 bookings, totalling ₹ 4,600.00.
Trip Details	3	1 Completed (19 min 16 s), 1 In Progress and 1 Scheduled.
Payments	4	Total ₹ 6,400.00: Paid ₹ 1,400.00 (2 cash), Pending ₹ 5,000.00 (UPI ₹ 600.00, Card ₹ 4,400.00); 21.9% of the value is collected.
Driver Maintenances	3	Total ₹ 8,750.00: Completed ₹ 2,950.00, Scheduled ₹ 5,800.00; the scheduled service is past its date.

Table 8.1: Operational Snapshot from the Supplied Data

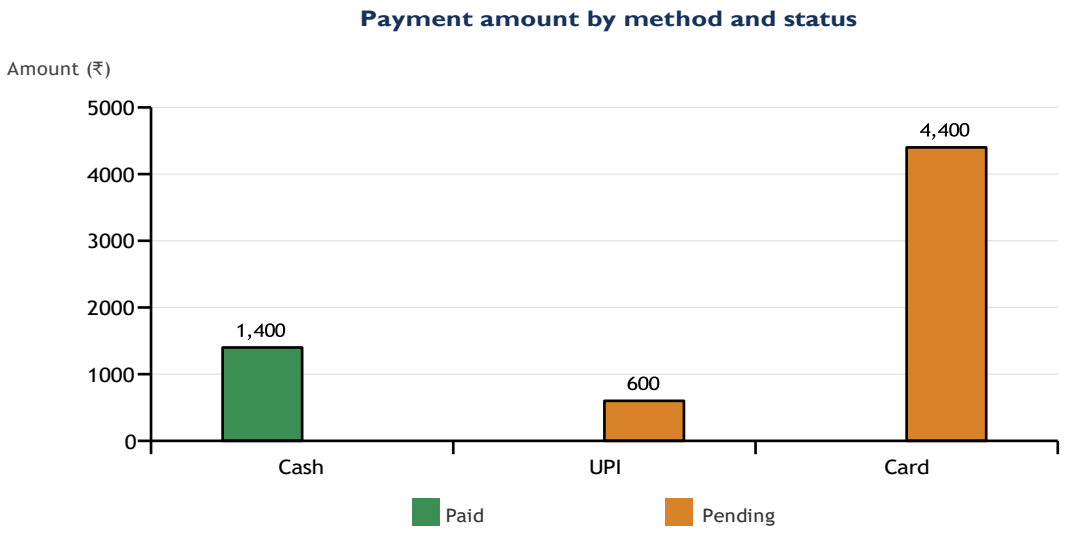


Figure 8.1: *Payment amount by method and status*

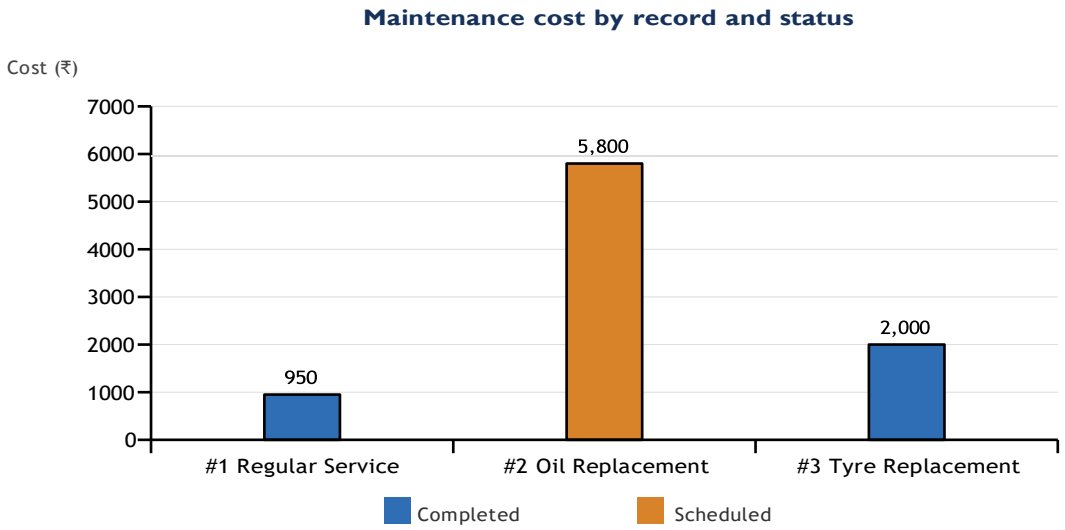


Figure 8.2: *Maintenance cost by record and status*

A traceable scenario can be followed through the records when they are related by date. The route from Maattuthavani Bus Stop to Madurai Airport was booked for 19-Oct-2026 at 15:30:00 and is Completed. A trip dated 19-Oct-2026 ran from 15:30:52 to 15:50:08, a duration of 19 minutes 16 seconds, and is Completed. A cash payment of ₹ 1,200.00 dated 19-Oct-2026 is Paid. The booking itself has no fare amount stored, which shows why the fare check proposed in Chapter 5 is needed.

8.2 Achievement of Objectives

Objective	Design / deliverable	Assessment
Linked data model	Seven forms for customers, drivers, vehicles, bookings, trips, payments and maintenance.	Forms evidenced by the supplied reports; lookups to be confirmed.
Booking capture	Bookings form with locations, date, time, status and fare.	Records exist; fare is missing in 3 of 5 bookings.
Allocation and trips	Driver and vehicle status; trip start and end times.	Status values exist; allocation rules need workflow evidence.
Payments	Payments form with method and status.	Records exist; reconciliation with fares is to be automated.
Maintenance	Maintenance form with type, date, cost and status.	Records exist; reminders are to be automated.
Visibility	Seven list reports; dashboard proposed.	Reports evidenced; dashboard to be built.

Table 8.2: Objective Achievement Mapping

8.3 Comparison with the Existing System

Area	Manual process	Proposed Taxi Management System
Booking	Phone calls and registers, often without fare.	Structured booking form with locations, time, fare and status.
Allocation	Dispatcher assigns from memory.	Only Available drivers and vehicles are selectable.
Trip tracking	Times noted by the driver, if at all.	Actual start and end times stored with trip status.
Payment	Entered later and not linked to a booking.	Payment per booking with method, status and reconciliation.
Maintenance	Informal service dates.	Scheduled records with cost, status and overdue flag.
Visibility	Manual consolidation.	Seven reports and a proposed dashboard.

Table 8.3: Comparison with the Existing System

8.4 Limitations of the Low-Code Approach

The solution depends on platform availability, plan capabilities and the correct configuration of permissions, workflows and schedules. The sample data set is small, so performance with large volumes is untested. Features such as live GPS tracking, automatic fare calculation, SMS or WhatsApp notifications and online payment need external services or additional development. Low-code does not remove the need for software testing, access-control review, data-quality checks or a deployment process.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

Visual form and report builders reduce repetitive coding, while Deluge provides control over allocation, reconciliation and reminder rules. The project therefore demonstrates a low-code approach rather than a purely no-code solution. A separate pro-code service may be justified in the future for requirements such as route optimisation, real-time vehicle tracking or surge-based fare calculation, but should be introduced only when platform-native capabilities are insufficient.

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

The Taxi Management System presents a structured low-code solution for managing taxi operations. Its forms connect customers, drivers, vehicles, bookings, trips, payments and maintenance, and its reports give the operator a view of each part of the business. Analysis of the supplied records shows the value of the planned controls: fares missing from bookings, booking and payment statuses that disagree, trips with inconsistent times, pending payments with no follow-up and a service that has passed its date.

The report follows the prescribed low-code project structure and documents the requirements, design, implementation, testing approach, security considerations and expected outcomes. Before the report is submitted as evidence of a live deployment, the workflows should be configured, the test cases executed, and the relevant application screenshots, execution logs and deployment details added.

9.2 Future Enhancements

- Configure and verify the proposed workflows, schedules and notification function in the live application.
- Add mandatory fare entry and automatic fare calculation based on distance and vehicle type.
- Add a Cancelled booking status and cancellation reasons.
- Build the dashboard, customer and driver pages and portal profiles with least-privilege access.
- Send booking and trip notifications by email, SMS or WhatsApp.
- Integrate an online payment gateway and UPI collection.
- Add live GPS tracking and a driver mobile app with trip start and end actions.
- Add ratings and feedback from customers and driver performance reports.
- Collect measurable results such as average trip duration, pending payment ageing and maintenance cost per vehicle.

9.3 Learning Outcomes and Career Relevance

This project provides practical experience in business process analysis, relational data modelling, form design, lookup relationships, report design, event-driven automation, Deluge scripting, scheduled workflows, data validation and test planning. These skills are directly relevant to roles in low-code development, business application development and process automation.

REFERENCES

[1] Zoho Corporation, “Zoho Creator Help Documentation,” Zoho. Available: https://www.zoho.com/creator/help/

[2] Zoho Corporation, “Deluge Help,” Zoho. Available: https://www.zoho.com/deluge/help/

[3] Zoho Corporation, “Workflow Rules in Zoho Creator,” Zoho Creator Help. Available: https://www.zoho.com/creator/help/

[4] Zoho Corporation, “Schedules in Zoho Creator,” Zoho Creator Help. Available: https://www.zoho.com/creator/help/

[5] Zoho Corporation, “Customer Portal in Zoho Creator,” Zoho Creator Help. Available: https://www.zoho.com/creator/help/

[6] Uber Technologies, “Uber – Request a ride.” Available: https://www.uber.com/

[7] ANI Technologies, “Ola – Book a cab.” Available: https://www.olacabs.com/

[8] OWASP Foundation, “Application Security Verification Standard (ASVS),” OWASP. Available: https://owasp.org/www-project-application-security-verification-standard/

APPENDIX A – FEATURE COVERAGE CHECKLIST

Feature area	Specified in project	Evidence to attach / verify
7 forms	Yes	Screenshots of each form in Creator and sample records.
Lookup relationships	Designed	Form builder screenshots and sample linked records.
Deluge validation and allocation scripts	Designed (not in supplied data)	Script screenshots and test outputs.
Form workflows	10 proposed	Workflow configuration screenshots and execution logs.
Scheduled workflows	2 proposed	Schedule configuration, activation and run history.
Notification function	tms.notifyCustomer (proposed)	Function screenshot and email delivery records.
Reports	7 list reports	Report screenshots with representative data.
Pages	4 proposed	Page screenshots for Dashboard, Book a Ride, My Bookings and Driver Duty.
Portal profiles	Admin, Dispatcher, Driver, Customer (proposed)	Portal profile screenshots and login test.
Deployment	Not evidenced in the supplied data	Published URL, release date and deployment checklist.

Table A.1: Feature Coverage Checklist

APPENDIX B – REPORTS AND PAGES CATALOGUE

B.1 Reports

- All Customers
- All Drivers
- All Vehicles
- All Bookings
- All Trip Details
- All Payments
- All Driver Maintenances

B.2 Pages (proposed)

- Dashboard
- Book a Ride
- My Bookings
- Driver Duty

B.3 Dashboard Indicators (proposed)

- Pending Bookings – count of bookings whose status is Pending.
- Available Drivers and Vehicles – count of drivers and vehicles whose status is Available.
- Pending Payments – total amount of payments whose status is Pending.
- Maintenance Due – count of Scheduled maintenance records whose service date is today or earlier.

B.4 Scheduled Monitoring

Frequency	Monitoring activities
Daily	Flag maintenance whose date has passed; list payments pending beyond the configured days; list Confirmed bookings without a driver or vehicle.
Weekly	Review pending and mismatched payments; review drivers who are Inactive; review upcoming maintenance.
Monthly	Review trip volume, revenue collected and pending, maintenance cost per vehicle and customer type mix.

Table B.1: Scheduled Monitoring

APPENDIX C – DELUGE LOGIC SUMMARY

The following pseudocode summaries describe the proposed behaviour of the scripts. They are design descriptions, not deployable Deluge source code, and the exact field link names must be confirmed in the live Creator application.

C.1 Booking defaults and fare check

On add, set Booking_Status to Pending and alert the user if a pickup location, drop location, booking date or pickup time is empty. On edit, when the new status is Confirmed, cancel the submission with an alert if Fare_Amount is empty.

C.2 Allocation

When a booking becomes Confirmed, read the selected driver and vehicle. Cancel with an alert if the driver status is not Available or the vehicle status is not Available. Otherwise set the driver status to On Trip and keep the vehicle linked to the booking.

C.3 Trip start and completion

On validate, cancel if Actual_End_Time is earlier than Actual_Start_Time. On start, set Trip_Status to In Progress. On completion, set Trip_Status to Completed, set the booking to Completed, set the driver to Available and call the notification function.

C.4 Payments

On validate, compare Amount with the Fare_Amount of the booking and alert on a mismatch. On success, when Payment_Status is Paid, store the payment date and keep the booking and payment statuses consistent. Count payments for the booking to avoid duplicates.

C.5 Maintenance and scheduled workflows

On success of a maintenance record, when the status is Scheduled keep the vehicle out of allocation on the service date, and when Completed release the vehicle. Maintenance Reminder: for each Scheduled record whose Service_Date is earlier than today, flag it overdue and email the administrator once. Pending Payment Follow-up: for each Pending payment older than the configured days, list it for the dispatcher.

C.6 Notification function

The function tms.notifyCustomer(bookingId, type, message) reads the customer of the booking and, when an email address exists, sends an email with the subject “Taxi Management System:” followed by the notification type.

APPENDIX D – USER MANUAL

D.1 General Access

1. Open the authorised Taxi Management System URL using a supported browser.
2. Sign in with the account provided.
3. Use the navigation menu to open the form or report appropriate to your role.
4. Enter mandatory fields, verify selections and save the record.
5. Use the status shown on reports to monitor progress; do not create duplicate records to retry a failed action.

D.2 Customer

6. Open Book a Ride and enter the pickup and drop locations, date and time.
7. Submit the booking and note its status.
8. Open My Bookings to follow the booking, trip and payment.

D.3 Dispatcher

9. Open the All Bookings report and review Pending bookings.
10. Enter the fare, confirm the booking and select an Available driver and vehicle.
11. Monitor trips in the All Trip Details report.
12. Record each payment with its method and status and follow up pending payments.

D.4 Driver

13. Open Driver Duty to see assigned trips.
14. Start the trip when the passenger is picked up and end it on arrival.
15. Keep the duty status current.

D.5 Administrator

16. Maintain customers, drivers (with licence numbers) and vehicles (with registration numbers).
17. Schedule maintenance with type, date and cost and mark it Completed after service.
18. Review reports and portal profile permissions periodically.
19. Test workflow changes in a safe environment before publishing.

D.6 Troubleshooting

Issue	Recommended check
Booking not saved	Confirm that pickup, drop, date and time are filled.
Cannot confirm booking	Check that a fare amount is entered.
Driver not available in the list	Check that the driver status is Available.
Trip not saved	Check that the end time is not earlier than the start time.
Payment and booking status differ	Check the payment status, the booking status and that the sync workflow is active.
No maintenance reminder	Check that the schedule is active and the record is Scheduled with a service date.

Table D.1: Troubleshooting

APPENDIX E – EVALUATION RUBRIC

Student Name	P Sharweshvar	**Register No.**	2116240701491
Project Title	Taxi Management System	**Date**	

S.No.	**Criterion**	**Indicators**	**Max. Marks**	**Marks Awarded**
1	Problem Definition & Requirement Analysis	Clarity of problem, measurable objectives, complete FR/NFR, use cases	10	
2	System Design	Architecture, ER/data model, form specifications, workflow design, permission matrix	10	
3	Forms, Reports & Pages	Variety of field types, lookups/subforms, report types, UI Builder dashboards	15	
4	Deluge Scripting & Workflows	Correctness and depth of scripts; form, schedule, approval, Blueprint, batch workflows	15	
5	Integration, RBAC & AI	Zoho/external integrations, connections, roles & permissions, Zia/analytics	10	
6	Testing, Deployment, Security & Governance	Test coverage, environments, optimisation, security controls, audit	10	
7	Live Demonstration	End-to-end working of the application across roles; feature checklist verified	15	
8	Project Report	Adherence to template and format, clarity, figures, references	5	
9	Viva-Voce	Understanding of the platform, design decisions and low-code concepts	10	
	TOTAL		**100**	

Table E.1: Evaluation Rubric for the Individual Project

Level	**% of Max. Marks**	**Descriptor**
Excellent	90–100	Feature implemented completely, correctly and beyond the basic requirement; student explains design choices confidently.
Good	70–89	Feature implemented correctly with minor gaps; explanation adequate.
Satisfactory	50–69	Feature partially implemented or works only for the main path; limited explanation.
Needs Improvement	< 50	Feature missing, non-functional or not understood by the student.

Table E.2: Performance Level Descriptors

Signature of Internal Examiner

Name: _____

Signature of External Examiner

Name: _____

APPENDIX F – VIVA-VOCE RECORD SHEET

Student Name	P Sharweshvar	**Register No.**	2116240701491
Department / Section	Computer Science And Engineering	**Date of Viva**	
Application URL Verified	Yes / No	**Demo Duration**	_____min

Q.No.	Question / Topic Asked	Unit	Response (E / G / S / NI)	Remarks
1				
2				
3				
4				
5				
6				
7				
8				

Table F.1: Viva-Voce Questions and Responses

E – Excellent G – Good S – Satisfactory NI – Needs Improvement

Component	Max. Marks	Marks Awarded
Criteria 1–8 (Appendix E)	90	
Viva-Voce	10	
TOTAL	**100**	

Table F.2: Marks Summary

Examiners' Remarks:

_____

INTERNAL EXAMINER
Name: _____

EXTERNAL EXAMINER
Name: _____